DBMS Lab: SQL Joins and Transactions

Use the following tables:

- Employees
- Departments
- Projects
- Customers
- Orders
- Products
- Order_Details

```
mysql> create database dbms_lab2;
Query OK, 1 row affected (0.04 sec)

mysql> use dbms_lab2;
Database changed
mysql> create table employees(employee_id int primary key auto_increment,employee_name varchar(100),employee_salary int);
Query OK, 0 rows affected (0.03 sec)

mysql> create table departments(department_id int primary key auto_increment,department_name varchar(120));
Query OK, 0 rows affected (0.18 sec)

mysql> create table projects(project_id int primary key auto_increment,project_name varchar(200));
Query OK, 0 rows affected (0.03 sec)

mysql> create table customers(customer_id int primary key auto_increment, customer_name varchar(120),city varchar(50));
Query OK, 0 rows affected (0.03 sec)

mysql> create table orders(order_id int primary key auto_increment,order_date date);
Query OK, 0 rows affected (0.06 sec)

mysql> create table products(product_id int primary auto_increment,product_name varchar(150),product_price decimal(10,2));
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the righ
t syntax to use near 'auto_increment,product_name varchar(150),product_price decimal(10,2))' at line 1
mysql> create table products(product_id int primary key auto_increment,product_name varchar(150),product_price decimal(10,2));
Query OK, 0 rows affected (0.06 sec)

mysql> create table order_details(order_details_id int primary key auto_increment, order_id int, product_id int, quantity int,foreign
 key(order_id) references orders(order_id), foreign key(product_id) references products(product_id);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the righ
t syntax to use near '' at line 1
mysql> create table order_details(order_details_id int primary key auto_increment, order_id int, product_id int, quantity int,foreign
 key(order_id) references orders(order_id), foreign key(product_id) references products(product_id));
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> insert into employees(employee_name,employee_salary) values('Rahul',30000.00),('Harini',20000.00),('Nithya',60000.00),('Sai',9
0000.00);
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> insert into departments(department_name) values('cse'),('ece'),('csit'),('aids');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> insert into projects(project_name) values('Mobile app'),('website devlopment'),('payroll system'),('Budget analysis');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> insert into customers(customer_name,city) values('Priya','Delhi'),('Keerthi','Hyd'),('Suresh','Goa'),('Arjun','Chennai');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> insert into orders(order_date) values('2026-03-18'),('2026-05-12'),('2026-012-12'),('2026-04-22');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0
       insert into products(product_name,product_price) values('laptop', 70000.00),('Mobile',15000.00),('Speaker',50000.00),('T.V',90
000.00);
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> insert into order_details(order_id,product_id,quantity) values(1,1,3),(2,2,2),(3,3,5),(4,4,1);
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> select * from employees;
+-------------+---------------+-----------------+
| employee_id | employee_name | employee_salary |
+-------------+---------------+-----------------+
|           1 | Rahul         |           30000 |
|           2 | Harini        |           20000 |
|           3 | Nithya        |           60000 |
|           4 | Sai           |           90000 |
+-------------+---------------+-----------------+
4 rows in set (0.00 sec)

mysql> select * from departments;
+---------------+-----------------+
| department_id | department_name |
+---------------+-----------------+
|             1 | cse             |
|             2 | ece             |
|             3 | csit            |
|             4 | aids            |
+---------------+-----------------+
4 rows in set (0.00 sec)

mysql> select * from projects;
+------------+--------------------+
| project_id | project_name       |
+------------+--------------------+
|          1 | Mobile app         |
|          2 | website devlopment |
|          3 | payroll system     |
|          4 | Budget analysis    |
+------------+--------------------+
4 rows in set (0.00 sec)
```

```
mysql> select * from customers;
+-------------+---------------+---------+
| customer_id | customer_name | city    |
+-------------+---------------+---------+
|           1 | Priya         | Delhi   |
|           2 | Keerthi       | Hyd     |
|           3 | Suresh        | Goa     |
|           4 | Arjun         | Chennai |
+-------------+---------------+---------+
4 rows in set (0.00 sec)

mysql> select * from orders;
+----------+------------+
| order_id | order_date |
+----------+------------+
|        1 | 2026-03-18 |
|        2 | 2026-05-12 |
|        3 | 2026-12-12 |
|        4 | 2026-04-22 |
+----------+------------+
4 rows in set (0.00 sec)

mysql> select * from products;
+------------+--------------+---------------+
| product_id | product_name | product_price |
+------------+--------------+---------------+
|          1 | laptop       |      70000.00 |
|          2 | Mobile       |      15000.00 |
|          3 | Speaker      |      50000.00 |
|          4 | T.V          |      90000.00 |
+------------+--------------+---------------+
4 rows in set (0.00 sec)
```

```
mysql> select * from order_details;
+-----------------+----------+------------+----------+
| order_details_id | order_id | product_id | quantity |
+-----------------+----------+------------+----------+
|               1 |        1 |          1 |        3 |
|               2 |        2 |          2 |        2 |
|               3 |        3 |          3 |        5 |
|               4 |        4 |          4 |        1 |
+-----------------+----------+------------+----------+
4 rows in set (0.00 sec)
```

DML Operations:

1. Insert a new employee into the Employees table.

```
mysql> insert into employees values('Rohan',80000);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> insert into employees values(5,'Rohan',80000);
Query OK, 1 row affected (0.04 sec)
```

2. Update the salary of employees working in the CSE department by 10%.

```
mysql> update employees e join departments d on e.department_id=d.department_id set e.employee_salary=e.employee_salary*1.10 where d.
department_name='cse';
Query OK, 2 rows affected (0.05 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

3. Delete employees whose salary is below ₹30,000.

```
mysql> delete from employees where employee_salary<30000;
Query OK, 1 row affected (0.04 sec)
```

4. Display all employee details.

```
mysql> select * from employees;
+-------------+---------------+-----------------+---------------+
| employee_id | employee_name | employee_salary | department_id |
+-------------+---------------+-----------------+---------------+
|           1 | Rahul         |           33000 |             1 |
|           3 | Nithya        |           60000 |          NULL |
|           4 | Sai           |           90000 |             3 |
|           5 | Rohan         |           80000 |             3 |
+-------------+---------------+-----------------+---------------+
4 rows in set (0.00 sec)
```

JOIN Queries:

5. Display employee names along with department names.

```
mysql> select employee_name,department_name from employees e inner join departments d on e.department_id=d.department_id;
+---------------+-----------------+
| employee_name | department_name |
+---------------+-----------------+
| Rahul         | cse             |
| Sai           | csit            |
| Rohan         | csit            |
+---------------+-----------------+
3 rows in set (0.04 sec)
```

6. Display employees who do not belong to any department.

```
mysql> select e.*
    -> from employees e
    -> left join departments d
    -> on e.department_id=d.department_id
    -> where d.department_id is null;
+-------------+---------------+-----------------+---------------+
| employee_id | employee_name | employee_salary | department_id |
+-------------+---------------+-----------------+---------------+
|           3 | Nithya        |           60000 |          NULL |
+-------------+---------------+-----------------+---------------+
1 row in set (0.04 sec)
```

7. Display all departments even if they have no employees.

```
mysql> SELECT d.*, e.employee_name
    -> FROM Departments d
    -> LEFT JOIN Employees e
    -> ON d.department_id = e.department_id;
+---------------+-----------------+---------------+
| department_id | department_name | employee_name |
+---------------+-----------------+---------------+
|             1 | cse             | Rahul         |
|             2 | ece             | NULL          |
|             3 | csit            | Rohan         |
|             3 | csit            | Sai           |
|             4 | aids            | NULL          |
+---------------+-----------------+---------------+
5 rows in set (0.00 sec)
```

8. Display employee and manager names using SELF JOIN.

```
mysql> alter table employees add manager_id int;
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> update employees set manager_id=3 where employee_id in (1,2);
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update employees set manager_id=3 where employee_id in (4,5);
Query OK, 2 rows affected (0.04 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> update employees set manager_id=1 where employee_id in (4,5);
Query OK, 2 rows affected (0.04 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> select e.employee_name as employee,
    -> m.employee_name as manager
    -> from employees e
    -> left join employees m
    -> on e.manager_id=m.employee_id;
+----------+---------+
| employee | manager |
+----------+---------+
| Rahul    | Nithya  |
| Nithya   | NULL    |
| Sai      | Rahul   |
| Rohan    | Rahul   |
+----------+---------+
4 rows in set (0.00 sec)

mysql> select * from employees;
+-------------+---------------+-----------------+---------------+------------+
| employee_id | employee_name | employee_salary | department_id | manager_id |
+-------------+---------------+-----------------+---------------+------------+
|           1 | Rahul         |           33000 |             1 |          3 |
|           3 | Nithya        |           60000 |          NULL |       NULL |
|           4 | Sai           |           90000 |             3 |          1 |
|           5 | Rohan         |           80000 |             3 |          1 |
+-------------+---------------+-----------------+---------------+------------+
4 rows in set (0.00 sec)
```

Aggregate Functions:

9. Find the total number of employees.

```
mysql> select count(*) as total_employees from employees;
+-----------------+
| total_employees |
+-----------------+
|               4 |
+-----------------+
1 row in set (0.01 sec)
```

10. Find department-wise average salary.

```
mysql> select d.department_name, avg(e.employee_salary) as avg_salary from departments d join employees e on d.department_id=e.depart
ment_id group by d.department_id,d.department_name;
+-----------------+------------+
| department_name | avg_salary |
+-----------------+------------+
| cse             | 33000.0000 |
| csit            | 85000.0000 |
+-----------------+------------+
2 rows in set (0.01 sec)
```

11. Display departments where average salary exceeds ₹60,000.

```
mysql> select d.department_name,avg(e.employee_salary) as avg_salary from departments d join employees e on d.department_id=e.departm
ent_id group by d.department_id,d.department_name having avg(e.employee_salary)>60000;
+-----------------+------------+
| department_name | avg_salary |
+-----------------+------------+
| csit            | 85000.0000 |
+-----------------+------------+
1 row in set (0.01 sec)
```

12. Display the highest and lowest salary.

```
mysql> select max(employee_salary) as high_salary, min(employee_salary) as lowest_salary from employees;
+-------------+--------------+
| high_salary | lowest_salary |
+-------------+--------------+
|       90000 |        33000 |
+-------------+--------------+
1 row in set (0.04 sec)
```

Subqueries:

13. Display employees earning more than the average salary.

```
mysql> select *
    -> from employees
    -> where employee_salary>( select avg(employee_salary) from employees);
+-------------+---------------+-----------------+---------------+------------+
| employee_id | employee_name | employee_salary | department_id | manager_id |
+-------------+---------------+-----------------+---------------+------------+
|           4 | Sai           |           90000 |             3 |          1 |
|           5 | Rohan         |           80000 |             3 |          1 |
+-------------+---------------+-----------------+---------------+------------+
2 rows in set (0.04 sec)
```

14. Find employees working in the same department as Rahul.

```
mysql> select *  from employees
    -> where department_id = ( select department_id
    -> from employees where employee_name='Rahul');
+-------------+---------------+-----------------+---------------+------------+
| employee_id | employee_name | employee_salary | department_id | manager_id |
+-------------+---------------+-----------------+---------------+------------+
|           1 | Rahul         |           33000 |             1 |          3 |
+-------------+---------------+-----------------+---------------+------------+
1 row in set (0.00 sec)
```

15. Display the employee receiving the highest salary.

```
mysql> select employee_name,employee_salary
    -> from employees
    -> where employee_salary =( select max(employee_salary) from employees);
+---------------+-----------------+
| employee_name | employee_salary |
+---------------+-----------------+
| Sai           |           90000 |
+---------------+-----------------+
1 row in set (0.00 sec)
```

16. Display departments having more than five employees.

```
mysql> select d.department_name, count(e.employee_id) as employee_count
    -> from departments d
    -> join employees e
    -> on d.department_id=e.department_id
    -> group by d.department_id,d.department_name
    -> having count(e.employee_id) > 5;
Empty set (0.04 sec)
```

17. Display employees earning above their department's average salary.

```
mysql> select e.employee_name,e.employee_salary,d.department_name from( select employee_name,employee_salary,department_id, avg(emplo
yee_salary) over (partition by department_id) as department_avg from employees ) e join departments d on e.department_id=d.department
_id where e.employee_salary > e.department_avg;
+---------------+-----------------+-----------------+
| employee_name | employee_salary | department_name |
+---------------+-----------------+-----------------+
| Sai           |           90000 | csit            |
+---------------+-----------------+-----------------+
1 row in set (0.05 sec)
```

18. Find customers who have never placed an order.

```
mysql> update orders
    -> set customer_id = 1 where order_id=1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update orders
    -> set customer_id = 3 where order_id=3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from orders;
+----------+------------+-------------+
| order_id | order_date | customer_id |
+----------+------------+-------------+
|        1 | 2026-03-18 |           1 |
|        2 | 2026-05-12 |        NULL |
|        3 | 2026-12-12 |           3 |
|        4 | 2026-04-22 |        NULL |
+----------+------------+-------------+
4 rows in set (0.00 sec)

mysql> select c.* from customers c
    -> left join orders o
    -> on c.customer_id=o.customer_id
    -> where o.order_id is null;
+-------------+---------------+---------+
| customer_id | customer_name | city    |
+-------------+---------------+---------+
|           2 | Keerthi       | Hyd     |
|           4 | Arjun         | Chennai |
+-------------+---------------+---------+
2 rows in set (0.00 sec)
```

19. Display the top-selling product.

```
mysql> select * from products;
+------------+--------------+---------------+
| product_id | product_name | product_price |
+------------+--------------+---------------+
|          1 | laptop       |      70000.00 |
|          2 | Mobile       |      15000.00 |
|          3 | Speaker      |      50000.00 |
|          4 | T.V          |      90000.00 |
+------------+--------------+---------------+
4 rows in set (0.01 sec)

mysql> select * from order_details;
+------------------+----------+------------+----------+
| order_details_id | order_id | product_id | quantity |
+------------------+----------+------------+----------+
|                1 |        1 |          1 |        3 |
|                2 |        2 |          2 |        2 |
|                3 |        3 |          3 |        5 |
|                4 |        4 |          4 |        1 |
+------------------+----------+------------+----------+
4 rows in set (0.01 sec)

mysql> select p.product_name, sum(od.quantity) as total_sold
    -> from products p
    -> join order_details od
    -> on p.product_id=od.product_id
    -> group by p.product_id,p.product_name
    -> order by total_sold desc
    -> limit 1;
+--------------+------------+
| product_name | total_sold |
+--------------+------------+
| Speaker      |          5 |
+--------------+------------+
1 row in set (0.04 sec)
```

20. Find departments with the maximum number of employees.

```
mysql> select d.department_name, count(e.employee_id) as employee_count
    -> from departments d
    -> join employees e
    -> on d.department_id=e.department_id
    -> group by d.department_id,d.department_name
    -> order by employee_count desc;
+-----------------+----------------+
| department_name | employee_count |
+-----------------+----------------+
| csit            |              2 |
| cse             |              1 |
+-----------------+----------------+
2 rows in set (0.00 sec)
```

21. Rank customers based on total purchase amount.

```
mysql> SELECT c.customer_name,
    ->        SUM(od.quantity * p.product_price) AS total_purchase,
    ->        RANK() OVER (ORDER BY SUM(od.quantity * p.product_price) DESC) AS customer_rank
    -> FROM customers c
    -> JOIN orders o ON c.customer_id=o.customer_id
    -> JOIN order_details od ON o.order_id=od.order_id
    -> JOIN products p ON od.product_id=p.product_id
    -> GROUP BY c.customer_id,c.customer_name;
+--------------+---------------+--------------+
| customer_name | total_purchase | customer_rank |
+--------------+---------------+--------------+
| Suresh        |      250000.00 |            1 |
| Priya         |      210000.00 |            2 |
+--------------+---------------+--------------+
2 rows in set (0.01 sec)
```

22. Display employees whose salary is greater than their manager's salary.

```
mysql> SELECT e.employee_name,m.employee_name
    -> FROM Employees e
    -> JOIN Employees m
    -> ON e.manager_id=m.employee_id
    -> WHERE e.employee_salary>m.employee_salary;
+---------------+---------------+
| employee_name | employee_name |
+---------------+---------------+
| Sai           | Rahul         |
| Rohan         | Rahul         |
+---------------+---------------+
2 rows in set (0.00 sec)
```

23. Find duplicate email addresses using GROUP BY and HAVING.

```
mysql> select * from employees;
+-------------+--------------+----------------+--------------+------------+---------------+
| employee_id | employee_name | employee_salary | department_id | manager_id | email         |
+-------------+--------------+----------------+--------------+------------+---------------+
|           1 | Rahul        |          33000 |            1 |          3 | abc@gmail.com |
|           3 | Nithya       |          60000 |         NULL |       NULL | abc@gmail.com |
|           4 | Sai          |          90000 |            3 |          1 | NULL          |
|           5 | Rohan        |          80000 |            3 |          1 | NULL          |
+-------------+--------------+----------------+--------------+------------+---------------+
4 rows in set (0.00 sec)
```

```
mysql> ALTER TABLE Employees
    -> ADD email VARCHAR(100);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> UPDATE Employees SET email='a@gmail.com' WHERE employee_id=1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Employees SET email='b@gmail.com' WHERE employee_id=2;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> UPDATE Employees SET email='abc@gmail.com' WHERE employee_id=3;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Employees SET email='abc@gmail.com' WHERE employee_id=1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Employees SET email='b@gmail.com' WHERE employee_id=2;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> SELECT email, COUNT(*)
    -> FROM Employees
    -> GROUP BY email
    -> HAVING COUNT(*) > 1;
+---------------+----------+
| email         | COUNT(*) |
+---------------+----------+
| abc@gmail.com |        2 |
| NULL          |        2 |
+---------------+----------+
2 rows in set (0.00 sec)
```